

10 MC-Aufgaben mit Dimensionen

Thema: Abstrakte Klassen in Java

Aufgabe 1 — A1: Faktisch / Erinnern

Welche Aussagen zu abstrakten Klassen in Java sind richtig?

- A. Eine abstrakte Klasse wird mit dem Schlüsselwort `abstract` deklariert.
 - B. Von einer abstrakten Klasse können direkt Objekte erzeugt werden.
 - C. Eine abstrakte Klasse kann gemeinsame Attribute und Methoden enthalten.
 - D. Eine abstrakte Klasse kann abstrakte Methoden enthalten.
 - E. Abstrakte Klassen dienen oft als Oberklassen für spezialisierte Unterklassen.
-

Aufgabe 2 — A1/A2: Faktisch / Erinnern-Verstehen

Welche Aussagen zu abstrakten Methoden sind richtig?

- A. Eine abstrakte Methode besitzt keinen Methodenrumpf.
- B. Eine abstrakte Methode muss in einer konkreten Unterklasse umgesetzt werden.
- C. Abstrakte Methoden dürfen in jeder beliebigen konkreten Klasse vorkommen.
- D. Die Umsetzung einer abstrakten Methode in der Unterklasse wird

oft mit `@Override` gekennzeichnet.

E. Eine abstrakte Methode gibt gemeinsames Verhalten nur als Vorgabe vor.

Aufgabe 3 — A2: Faktisch / Verstehen

Welche Aussagen zur Klasse `Krokodil` sind richtig?

- A. `Krokodil` ist eine abstrakte Oberklasse.
 - B. `Krokodil` besitzt das Attribut `name`.
 - C. `Krokodil` enthält die Methode `getName()`.
 - D. `Krokodil` implementiert `gibArtInfo()` bereits vollständig.
 - E. `Krokodil` enthält mit `beschreibe()` eine konkrete Methode.
-

Aufgabe 4 — B2: Konzeptionell / Verstehen

Welche Aussagen zur Rolle der abstrakten Oberklasse `Krokodil` sind richtig?

- A. Sie bündelt gemeinsame Merkmale aller Krokodile.
 - B. Sie verhindert unnötige Mehrfachimplementationen gemeinsamer Methoden.
 - C. Sie soll direkt als normales Objekt erzeugt werden.
 - D. Sie gibt mit `gibArtInfo()` eine verpflichtende Struktur für Unterklassen vor.
 - E. Sie unterstützt die Trennung zwischen allgemeinem und spezialisiertem Verhalten.
-

Aufgabe 5 — A1/C2: Faktisch-prozedural / Erinnern-Verstehen

Welche Aussagen zu `Leistenkrokodil` und `Nilkrokodil` sind richtig?

- A. Beide Klassen erben von `Krokodil`.
- B. Beide Klassen rufen im Konstruktor mit `super(name)` den Konstruktor der Oberklasse auf.
- C. Beide Klassen implementieren `gibArtInfo()` auf eigene Weise.
- D. Beide Klassen müssen selbst das Attribut `name` erneut deklarieren.
- E. Beide Klassen sind konkrete Unterklassen.

Aufgabe 6 — B2/C2: Konzeptionell-prozedural / Verstehen

Welche Aussagen zum Konstruktoraufruf mit `super(...)` sind richtig?

- A. `super(name)` ruft den Konstruktor der Oberklasse auf.
- B. Damit kann das geerbte Attribut `name` korrekt initialisiert werden.
- C. `super(name)` ist ein Aufruf einer frei wählbaren Methode der Oberklasse.
- D. Der Aufruf von `super(...)` steht im Konstruktor inhaltlich am Anfang der Initialisierung.
- E. Ohne passenden Oberklassenkonstruktor würde der Code so nicht funktionieren.

Aufgabe 7 — C3: Prozedural / Anwenden

Welche Aussagen zur Testklasse `Abstrakt` sind richtig?

- A. In `main` werden Objekte konkreter Unterklassen erzeugt.
- B. Es werden gemeinsame Methoden der Oberklasse aufgerufen.
- C. Es werden unterklassenspezifische Implementierungen von `gibArtInfo()` genutzt.
- D. Das Testprogramm zeigt, dass unterschiedliche Unterklassen dieselbe Methode verschieden umsetzen können.
- E. In der Testklasse wird ein Objekt der abstrakten Klasse `Krokodil` direkt erzeugt.

Aufgabe 8 — B3/C3: Konzeptionell-prozedural / Anwenden

Welche Aussagen zur Klasse `Künstler` mit den Unterklassen `Maler` und `Sänger` sind richtig?

- A. `Künstler` ist als abstrakte Oberklasse modelliert.
- B. `kunstErstellen()` ist in `Künstler` abstrakt vorgegeben.
- C. `Maler` und `Sänger` überschreiben `kunstErstellen()` mit eigener Ausgabe.
- D. `Maler` und `Sänger` können direkt Objekte von `Künstler` erzeugen.
- E. Die Klassenhierarchie zeigt, dass dieselbe Methode je nach Unterklasse anders ausgeführt werden kann.

Aufgabe 9 — B4: Konzeptionell / Analysieren

Analysieren Sie die gegebene Klassenstruktur. Welche Aussagen sind richtig?

- A. Die Beispiele mit `Krokodil` sowie mit `Künstler` zeigen jeweils eine abstrakte Oberklasse mit konkreten Unterklassen.
- B. In beiden Beispielen wird ein allgemeines Modell durch spezialisierte Klassen konkretisiert.
- C. Die Methode `gibArtInfo()` im `Krokodil`-Beispiel und `kunstErstellen()` im `Künstler`-Beispiel erfüllen dieselbe didaktische Funktion.
- D. Die Unterklassen unterscheiden sich nur im Namen, aber nicht im Verhalten.
- E. Beide Beispiele bereiten das Verständnis von Polymorphie vor.

Aufgabe 10 — B5/C5: Konzeptionell-prozedural / Beurteilen

Welche Aussagen zur Modellierung und zum Stil sind sinnvoll?

- A. Eine abstrakte Oberklasse ist sinnvoll, wenn gemeinsame Eigenschaften zentral gebündelt werden sollen.
- B. Die Trennung in Oberklasse und konkrete Unterklassen erhöht die Übersichtlichkeit des Codes.
- C. Abstrakte Methoden sind sinnvoll, wenn Unterklassen unterschiedliches Verhalten verpflichtend umsetzen sollen.

D. Es ist guter Stil, eine abstrakte Klasse direkt zu instanziiieren, wenn man nur schnell testen möchte.

E. Die Beispiele sind didaktisch geeignet, um Vererbung, Spezialisierung und Methodenüberschreibung zu verdeutlichen.

Lösungen mit Erklärungen

Lösung zu Aufgabe 1 — A1

Richtige Antworten: A, C, D, E

Erklärung:

- **A richtig:** Abstrakte Klassen werden mit `abstract` deklariert.
 - **B falsch:** Von abstrakten Klassen können keine direkten Objekte erzeugt werden.
 - **C richtig:** Sie dürfen gemeinsame Attribute und konkrete Methoden enthalten.
 - **D richtig:** Sie dürfen auch abstrakte Methoden enthalten.
 - **E richtig:** Genau dafür werden sie typischerweise genutzt.
-

Lösung zu Aufgabe 2 — A1/A2

Richtige Antworten: A, B, D, E

Erklärung:

- **A richtig:** Abstrakte Methoden haben keinen Methodenrumpf.
 - **B richtig:** Konkrete Unterklassen müssen sie umsetzen.
 - **C falsch:** Eine abstrakte Methode darf nicht einfach in einer beliebigen konkreten Klasse ohne passende abstrakte Klassenstruktur stehen.
 - **D richtig:** `@Override` kennzeichnet die Überschreibung in der Unterklasse.
 - **E richtig:** Die Methode gibt nur die Struktur oder Pflicht vor, nicht die konkrete Ausführung.
-

Lösung zu Aufgabe 3 — A2

Richtige Antworten: A, B, C, E

Erklärung:

- **A richtig:** `Krokodil` ist als `abstract class` definiert.
- **B richtig:** Das Attribut `name` ist vorhanden.
- **C richtig:** `getName()` liefert den Namen zurück.
- **D falsch:** `gibArtInfo()` ist abstrakt und gerade **nicht** vollständig implementiert.
- **E richtig:** `beschreibe()` ist eine konkrete Methode der Oberklasse.

Lösung zu Aufgabe 4 — B2

Richtige Antworten: A, B, D, E

Erklärung:

- **A richtig:** Gemeinsame Merkmale wie `name` oder `beschreibe()` werden in der Oberklasse gebündelt.
- **B richtig:** So muss gemeinsamer Code nicht mehrfach in Unterklassen geschrieben werden.
- **C falsch:** Eine abstrakte Klasse ist nicht für direkte Instanziierung gedacht.
- **D richtig:** Mit `gibArtInfo()` wird eine verpflichtende Struktur vorgegeben.
- **E richtig:** Genau darin liegt der didaktische Kern abstrakter Klassen.

Lösung zu Aufgabe 5 — A1/C2

Richtige Antworten: A, B, C, E

Erklärung:

- **A richtig:** Beide Klassen `extends Krokodil`.
- **B richtig:** Beide rufen im Konstruktor `super(name)` auf.

- **C richtig:** Beide setzen `gibArtInfo()` unterschiedlich um.
 - **D falsch:** Das Attribut `name` wird geerbt und nicht erneut deklariert.
 - **E richtig:** Beide sind konkrete, also instanziiierbare Unterklassen.
-

Lösung zu Aufgabe 6 — B2/C2

Richtige Antworten: A, B, D, E

Erklärung:

- **A richtig:** `super(name)` ruft den Konstruktor der Oberklasse auf.
 - **B richtig:** Dadurch wird das geerbte Attribut initialisiert.
 - **C falsch:** `super(...)` ist hier kein beliebiger Methodenaufruf, sondern ein Konstruktoraufruf.
 - **D richtig:** Inhaltlich startet damit die Initialisierung der Oberklasse.
 - **E richtig:** Fehlt ein passender Oberklassenkonstruktor, gibt es einen Fehler.
-

Lösung zu Aufgabe 7 — C3

Richtige Antworten: A, B, C, D

Erklärung:

- **A richtig:** In `main` werden etwa `Leistenkrokodil` und

Nilkrokodil erzeugt.

- **B richtig:** `beschreibe()` oder `getName()` stammen aus der Oberklasse.
- **C richtig:** `gibArtInfo()` wird je nach Unterklasse unterschiedlich ausgeführt.
- **D richtig:** Genau das zeigt das Testprogramm.
- **E falsch:** Ein Objekt von `Krokodil` direkt zu erzeugen wäre nicht zulässig.

Lösung zu Aufgabe 8 — B3/C3

Richtige Antworten: A, B, C, E

Erklärung:

- **A richtig:** `Künstler` ist als abstrakte Oberklasse gedacht.
- **B richtig:** `kunstErstellen()` ist dort abstrakt vorgegeben.
- **C richtig:** `Maler` und `Sänger` überschreiben die Methode.
- **D falsch:** Unterklassen erzeugen keine Objekte der Oberklasse; `Künstler` selbst ist abstrakt.
- **E richtig:** Dasselbe Methodenlabel führt je nach Unterklasse zu anderem Verhalten.

Lösung zu Aufgabe 9 — B4

Richtige Antworten: A, B, C, E

Erklärung:

- **A richtig:** Beide Beispiele folgen demselben Grundmuster.
 - **B richtig:** Das Allgemeine liegt in der Oberklasse, das Spezielle in den Unterklassen.
 - **C richtig:** Didaktisch erfüllen `gibArtInfo()` und `kunstErstellen()` dieselbe Funktion: Sie erzwingen konkrete Umsetzung in Unterklassen.
 - **D falsch:** Die Unterklassen unterscheiden sich gerade im Verhalten.
 - **E richtig:** Damit wird Polymorphie vorbereitet, auch wenn sie noch nicht maximal ausgebaut wird.
-

Lösung zu Aufgabe 10 — B5/C5

Richtige Antworten: A, B, C, E

Erklärung:

- **A richtig:** Das ist einer der Hauptgründe für abstrakte Oberklassen.
- **B richtig:** Die Struktur wird klarer und wartbarer.
- **C richtig:** Abstrakte Methoden erzwingen spezialisierte Implementierungen.
- **D falsch:** Eine abstrakte Klasse darf nicht direkt instanziiert werden.

- **E richtig:** Die Beispiele sind sehr gut geeignet, um das Prinzip verständlich zu machen.
-

Übersicht der Dimensionen

1. **A1** – Faktisch / Erinnern
 2. **A1/A2** – Faktisch / Erinnern-Verstehen
 3. **A2** – Faktisch / Verstehen
 4. **B2** – Konzeptionell / Verstehen
 5. **A1/C2** – Faktisch-prozedural / Erinnern-Verstehen
 6. **B2/C2** – Konzeptionell-prozedural / Verstehen
 7. **C3** – Prozedural / Anwenden
 8. **B3/C3** – Konzeptionell-prozedural / Anwenden
 9. **B4** – Konzeptionell / Analysieren
 10. **B5/C5** – Konzeptionell-prozedural / Beurteilen
-